Limitations of Algorithmic Power

(also covered in Levitin chapter 11)

Analysis of Algorithms

- How good is the algorithm?
  - Correctness
  - Time efficiency
  - Space efficiency
- Could there be a better algorithm?
  - Lower Bounds
  - Optimality
- Approaches:
  - Theoretical analysis
  - Empirical analysis
  - Visualisation

Best-, Average- and Worst-cases

- For some algorithms efficiency depends on type of input:
  - Worst case: W(n) – max over inputs of size n
  - Best case: B(n) – min over inputs of size n
  - Average case: A(n) – “average” over inputs of size n
    - Number of times the basic operation will be executed on typical input
    - NOT the average of worst and best case
    - INSTEAD expected number of ops considered as a random variable under some assumption about the probability distribution of all possible inputs of size n
    - most difficult to estimate, as need to make assumptions about the possible inputs of size n

Exercise: Sequential search

- Problem:
  - Given a list of *n* elements and a search key *K*, find an element equal to *K*, if any exists
- Algorithm:
  - Scan the list and compare successive elements with *K* until either a matching element is found (*successful search*) or the list is exhausted (*unsuccessful search*)
- Calculate the:
  - Worst case?
  - Best case?
  - Average case?

How good is your algorithm?

Finding an algorithm to solve a problem is great ...

but :

- some problems can't be solved by any algorithm
- others have an algorithm, but it is superpolynomial

An algorithm is superpolynomial if its complexity is worse than polynomial, i.e. worse than $O(n^K)$ for any integer K

Running Times:
polynomial (in blue) vs super-polynomial (in black)

Size:	10	20	30	40	50	60
n	.00001 second	.00002 second	.00003 second	.00004 second	.00005 second	.00006 second
n^2	.0001 second	.0004 second	.0009 second	.0016 second	.0025 second	.0036 second
n^3	.001 second	.008 second	.027 second	.064 second	.125 second	.216 second
n^5	.1 second	3.2 second	24.3 seconds	1.7 minutes	5.2 minutes	13.0 minutes
2^n	.001 second	1.0 second	17.9 minutes	12.7 days	35.7 years	36 600 years
3^n	.059 second	58 minutes	6.5 years	385 500 years	2×10^{10} years	1.3×10^{15} years

How good is your algorithm?

You've seen many **polynomial time algorithms** for problems that search for a solution among an **exponential population of possibilities**.

An algorithm that runs in superpolynomial time is all but useless in practice.

We will soon consider some problems where we can't seem to do better than superpolynomial time

What we'd like to know

If we have a polynomial time algorithm to solve a problem
is it the best possible algorithm for that problem?

If we don't have a polynomial time algorithm:
is it unsolvable ?
is it intractable ?
or are we just not clever enough ?

Algorithms are powerful but have their limits

A problem can be:

- **tractable**: solvable in polynomial (reasonable) time
- **intractable**: solvable only in super-polynomial time
- **unsolvable** (even if you had forever)

Some problems have been proven tractable, intractable or unsolvable, but for many problems, **we don't know**

For many important problems, we *suspect* they are intractable but we cannot prove this – however we can show that they all stand or fall together (if we could solve one in polynomial time, we can solve all of them in polynomial time)

Is there a polynomial-time algorithm that solves my problem?

Possible answers:

- Yes, because we already have one
- **No, because it can be proved that:**
  - **all algorithms take super-polynomial time**
  - **no algorithm exists at all to solve this problem**
- don't know
- don't know, but a solution can be checked in polynomial time
- don't know, but if such an algorithm were found, then it would provide a means of solving many other problems in polynomial time

I can’t find an efficient algorithm, I guess I’m just too dumb.





I can’t find an efficient algorithm, because no such algorithm is possible!



I can’t find an efficient algorithm, but neither can all these famous people!



What this section is about:

If we have a polynomial time algorithm: **is it the best possible algorithm for that problem?**

- lower bounds
- tight bounds and problems with algorithmic gaps
- devising lower bounds: adversary arguments, problem reductions, decision trees

If we don't have a polynomial time algorithm: **is the problem tractable or not? - COMPLEXITY THEORY**

- is it NP? (can we check a solution in polynomial time?)
- is it NP Complete? (is it one of the hardest NP problems?)
- some example NP Complete problems

Types of problems

Optimization problem: construct a solution that maximizes or minimizes some objective function

Decision problem: answer yes/no to a question

Many problems will have decision and optimization versions.

Eg: Travelling salesman problem

- *optimization*: find cycle of minimum weight

  given a solution, we ***cannot check***/verify this in polynomial time

- *decision*: Does a cycle of weight < *k* exist*?*

  given a solution, we ***can check*** it is indeed a solution in polynomial time